Lab 6: Design Analysis Report

Passport Appointment System

Subject: Software Engineering · **Assignment:** 5

Name	University Roll No.
Ankit Kumar	202401100200054
Ankit Patel	202401100200055
Ankush	202401100200057
Ankur Singh	202401100200056

1. Introduction

The Passport Appointment System is a web-based application that lets citizens book, reschedule and cancel appointments at a Passport Seva Kendra (PSK) or Post Office Passport Seva Kendra (POPSK). It replaces long physical queues with a slot-based scheduling model and gives officers a clear daily view of their workload.

This report analyses the design of the system. It covers the requirements the design must satisfy, the architecture, the structural and behavioural models, the data design, the design principles and patterns applied, and an evaluation of quality attributes and risks.

1.1 Objectives

- Allow applicants to register, log in and submit passport applications online.
- Allow applicants to pay fees and book an appointment in an available slot.
- Prevent double booking and overbooking of any centre slot.
- Let officers and admins verify documents and manage centres and slots.
- Send timely notifications (SMS or email) for confirmation, reminders and changes.

1.2 Scope

In scope: user management, application form handling, fee payment, appointment scheduling, document verification status, notifications and reporting. Out of scope: police verification workflow, passport printing and dispatch, and biometric hardware integration.

2. Stakeholders and Users

Actor	Role in the system
Applicant (Citizen)	Registers, fills the application, pays, books and manages appointments
Passport Officer	Views the daily schedule, verifies documents, marks appointment status
Administrator	Manages centres, working days, slot capacity, officers and holidays
Payment Gateway (external)	Processes online fee payment
Notification Service (external)	Sends SMS and email messages

3. Requirements Analysis

3.1 Functional Requirements

ID	Requirement
FR1	The system shall allow a new user to register and verify their email or mobile number.
FR2	The system shall authenticate users using ID and password with OTP support.
FR3	The applicant shall be able to fill in and save an application form as a draft.
FR4	The system shall calculate the fee based on application type (fresh, re-issue, Tatkal).
FR5	The system shall show available centres, dates and time slots.
FR6	The system shall allow booking only after successful fee payment.
FR7	The system shall allow rescheduling or cancelling within the permitted limits.
FR8	The system shall generate an appointment receipt with a reference number.

ID	Requirement
FR9	The officer shall be able to view appointments and update their status.
FR10	The admin shall be able to configure centres, slot capacity and holidays.
FR11	The system shall send confirmation and reminder notifications.

3.2 Non-Functional Requirements

Attribute	Requirement
Performance	Slot search should respond within 3 seconds under normal load.
Scalability	Must handle heavy concurrent booking when new slots are released.
Security	Encrypted transport (HTTPS), hashed passwords, role-based access, protection of personal data.
Availability	Target of 99.5 percent uptime during working hours.
Usability	Simple interface, usable on mobile, with clear error messages.
Reliability	No double booking even under concurrent requests.
Maintainability	Modular design so that rules (fees, limits) can change without major rewrites.

4. Use Case Analysis

4.1 Main Use Cases

- **Applicant:** Register, Login, Fill Application, Pay Fee, Book Appointment, Reschedule Appointment, Cancel Appointment, Download Receipt, Track Status.
- **Officer:** Login, View Daily Schedule, Verify Documents, Update Appointment Status.
- **Admin:** Manage Centres, Manage Slots, Manage Holidays, Manage Officers, Generate Reports.

4.2 Use Case Description: Book Appointment

Field	Description
Actor	Applicant
Precondition	Applicant is logged in and has a submitted application with paid fee
Main flow	1. Applicant selects Book Appointment. 2. System shows centres. 3. Applicant picks a centre and date. 4. System lists available slots. 5. Applicant selects a slot. 6. System locks the slot, confirms the booking and generates a receipt. 7. System sends a notification.
Alternate flow	If the chosen slot becomes full before confirmation, the system shows the remaining slots and asks the applicant to choose again.
Exception flow	If payment is not verified, the system blocks booking and prompts the applicant to complete payment.
Postcondition	Appointment is stored with status Scheduled and slot capacity is reduced by one.

5. Architectural Design

5.1 Chosen Architecture

The system follows a **layered (three-tier) architecture** with a client-server model:

1. **Presentation layer:** Web and mobile-friendly user interface for applicants, officers and admins.
2. **Business logic layer:** Services that implement registration, application handling, fee calculation, slot allocation, appointment management and notification triggering.
3. **Data layer:** A relational database storing users, applications, centres, slots, appointments and payments.

External systems (payment gateway, SMS and email service) are reached through adapter components in the business layer.

5.2 Major Components

Component	Responsibility
User Management Module	Registration, authentication, roles and profile
Application Module	Form capture, validation, draft saving, document upload
Payment Module	Fee calculation, gateway interaction, payment status
Scheduling Module	Slot generation, availability search, booking, rescheduling, cancellation
Notification Module	Template-based SMS and email delivery
Admin and Reporting Module	Centre and slot configuration, statistics

5.3 Justification

A layered architecture keeps concerns separate, makes each layer independently testable, and lets the user interface change without touching business rules. It is also simple enough for a project team of this size to build and maintain. A microservice style was considered but rejected because the added deployment and communication complexity is not justified for the expected scale of a single-organisation system.

6. Structural Design (Class Analysis)

6.1 Key Classes

Class	Main attributes	Main operations
User	userId, name, email, mobile, passwordHash, role	register(), login(), updateProfile()
Applicant (extends User)	dob, address	submitApplication(), viewStatus()
Officer (extends User)	officerId, centreId	viewSchedule(), verifyDocuments()
Admin (extends User)	adminId	manageCentre(), manageSlots()

Class	Main attributes	Main operations
Application	applicationId, type, status, createdOn	save(), submit(), calculateFee()
Payment	paymentId, amount, status, transactionRef	initiate(), verify(), refund()
Centre	centreId, name, city, address	getAvailableSlots()
Slot	slotId, date, startTime, endTime, capacity, booked	isAvailable(), reserve(), release()
Appointment	appointmentId, status, referenceNo	confirm(), reschedule(), cancel()
Notification	notificationId, channel, message, sentOn	send()

6.2 Relationships

- User is a generalisation of Applicant, Officer and Admin (inheritance).
- One Applicant has many Applications (1 to many).
- One Application has one Payment and at most one active Appointment.
- One Centre has many Slots, and one Slot has many Appointments up to its capacity.
- One Officer is assigned to one Centre.
- Appointment triggers one or more Notifications.

7. Behavioural Design

7.1 Sequence: Booking an Appointment

1. Applicant requests available slots from the interface.
2. Interface calls the Scheduling Service.
3. Scheduling Service queries the Slot table and returns open slots.
4. Applicant selects a slot.
5. Scheduling Service checks the Payment status through the Payment Service.
6. Scheduling Service reserves the slot inside a database transaction (reduces availability by one).
7. Appointment record is created with status Scheduled.
8. Notification Service sends a confirmation message.
9. Interface displays the receipt.

7.2 Appointment State Model

From state	Event	To state
Pending	Payment verified and slot chosen	Scheduled
Scheduled	Applicant reschedules	Rescheduled (new Scheduled slot)
Scheduled	Applicant cancels	Cancelled
Scheduled	Applicant visits and officer verifies	Completed
Scheduled	Applicant does not attend	Missed

8. Data Design

8.1 Main Tables

Table	Key fields
users	user_id (PK), name, email, mobile, password_hash, role
applications	application_id (PK), user_id (FK), type, status, created_on
payments	payment_id (PK), application_id (FK), amount, status, txn_ref
centres	centre_id (PK), name, city, address
slots	slot_id (PK), centre_id (FK), slot_date, start_time, end_time, capacity, booked_count
appointments	appointment_id (PK), application_id (FK), slot_id (FK), status, reference_no
notifications	notification_id (PK), appointment_id (FK), channel, message, sent_on

8.2 Design Decisions

- A **unique constraint on application_id for active appointments** ensures one active appointment per application.

- The slot reservation uses a **transaction with row-level locking** so that two applicants cannot take the last seat together.
- Passwords are stored only as salted hashes, and sensitive personal fields are encrypted at rest.
- Tables are normalised to third normal form to avoid redundancy.

9. User Interface Design

The interface follows these guidelines:

- A step-by-step flow (Register, Apply, Pay, Book, Receipt) with a visible progress indicator.
- A calendar view that greys out holidays and full dates.
- Inline validation with plain-language error messages.
- Responsive layout for phones, since many applicants use mobile devices.
- Accessibility support such as readable contrast, keyboard navigation and text labels.

10. Design Principles and Patterns Applied

10.1 Principles

Principle	How it is applied
High cohesion	Each module handles one clear responsibility (for example, scheduling logic is separate from notifications).
Low coupling	Modules communicate through defined service interfaces rather than directly sharing data.
Single Responsibility	Classes such as Payment and Slot each change for only one reason.
Open/Closed	New fee types or notification channels can be added without altering existing code.
Dependency Inversion	Business logic depends on abstractions (PaymentGateway, MessageSender) and not on concrete vendors.
Separation of concerns	The layered architecture divides presentation, logic and data.

10.2 Design Patterns

- **MVC (Model-View-Controller):** Organises the web layer.
- **Strategy:** Fee calculation varies by application type (fresh, re-issue, Tatkal).
- **Adapter:** Wraps third-party payment and SMS APIs behind common interfaces.
- **Observer:** The Notification Module reacts to appointment events (confirmed, rescheduled, cancelled).
- **Factory:** Creates the correct user object (Applicant, Officer, Admin) based on role.
- **Singleton:** Used for configuration and logging services.

11. Quality Attribute Analysis

Attribute	Design support	Possible weakness
Security	Role-based access, HTTPS, hashed passwords, input validation	Needs rate limiting and OTP to resist brute force and bot booking
Performance	Indexed slot queries, caching of centre data	Booking spikes when slots open may slow the database
Scalability	Stateless application servers behind a load balancer	Single database may become a bottleneck
Reliability	Transactional booking, payment verification before confirmation	Gateway downtime can block bookings
Maintainability	Modular layers and abstractions	Requires disciplined documentation and tests
Usability	Guided flow, mobile-friendly	Language support must be extended for regional users

12. Risk Analysis

Risk	Impact	Mitigation
Concurrent booking of the same slot	Double booking	Database transactions and row locking
Payment succeeds but booking fails	User dissatisfaction, refunds	Reconciliation job and automatic refund or retry

Risk	Impact	Mitigation
Bots grabbing slots	Genuine users miss slots	CAPTCHA, OTP and per-user limits
Personal data leakage	Legal and trust damage	Encryption, access logs, least-privilege roles
Notification failure	Missed appointments	Retry queue and fallback channel

13. Testing Considerations

- **Unit tests** for fee calculation, slot availability and state transitions.
- **Integration tests** for payment and notification adapters using mock services.
- **Concurrency tests** simulating many users booking the final slot.
- **Security tests** for authentication, authorisation and input validation.
- **Acceptance tests** based on the use cases in Section 4.

14. Strengths and Limitations of the Design

Strengths

- Clear separation of responsibilities and easy to extend.
- Strong protection against double booking.
- Use of standard patterns keeps the design understandable for new team members.

Limitations

- A single relational database limits horizontal scaling.
- Dependence on external payment and SMS vendors.
- No offline or assisted-booking channel for users without internet access.

15. Conclusion

The layered, modular design of the Passport Appointment System meets the stated functional and non-functional requirements. Applying cohesion, low coupling, SOLID principles and well-known patterns makes the system maintainable and extensible, while transactional slot booking and role-based security address its main reliability and safety concerns. Future work can include a read-replica or caching strategy for peak loads, multilingual support, and integration with the police verification workflow.

References

- Pressman, R. S., *Software Engineering: A Practitioner's Approach*.
- Sommerville, I., *Software Engineering*.
- Gamma, E. et al., *Design Patterns: Elements of Reusable Object-Oriented Software*.
- Passport Seva portal (Ministry of External Affairs, Government of India), for general process reference.